Group, Solve, Merge!

A decomposition heuristic for makespan-minimising multi-trip vehicle routing

Polikarpus Arya Pradhanika

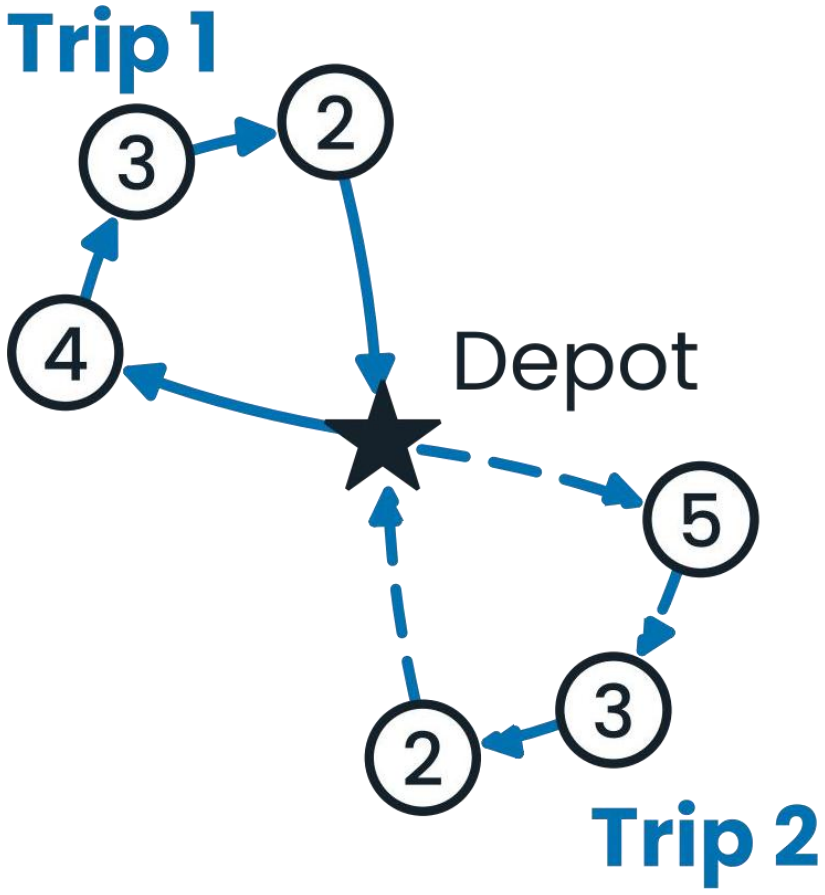
Ehime University

1 Motivation

Given a depot, customers who need goods, and a limited fleet of vehicles, we plan how to deliver.

With a limited fleet, one vehicle often makes several trips, and the day ends when the last vehicle is back.

Exact solvers are slow, so we need fast, quasi-optimal solutions.



One vehicle serves some customers, returns to the depot, then goes out again. Numbers are demands.

2 Problem

A depot and N customers, each at a known location with a known demand.

K vehicles, each carrying at most Q units, may run several trips from the depot.

Travel distance is the straight-line (Euclidean) distance between two points.

Goal: minimise the **makespan**, the time the last vehicle returns.

Notation

N	number of customers
i, j	customer indices
(x_i, y_i)	location of customer i
q_i	demand of customer i
$d_{i,j}$	$\sqrt{((x_i - x_j)^2 + (y_i - y_j)^2)}$
K	number of vehicles
Q	vehicle's capacity
$T(r)$	duration of trip r

3 Method

PRUNE

Line sweep keeps only short candidate edges

GROUP

Kruskal merges customers while demand $\leq Q$ and size $\leq X$

SOLVE

Held-Karp or Lin-Kernighan (LK) gives one tour per group

MERGE

Binary search on the makespan, pack trips onto vehicles

Notation

E	number of edges	LB	lower bound	X	maximum group size
G	number of groups after merging (one trip each)	UB	upper bound	T^*	makespan of final solution
R	set of trips, $\lvert R\rvert = G$				

$$E \leq N(N-1)\ /\ 2$$

$$UB = \Sigma_{r \in R}\ T(r)$$

$$LB = \max\{\ T(r) \mid r \in R\ \}$$

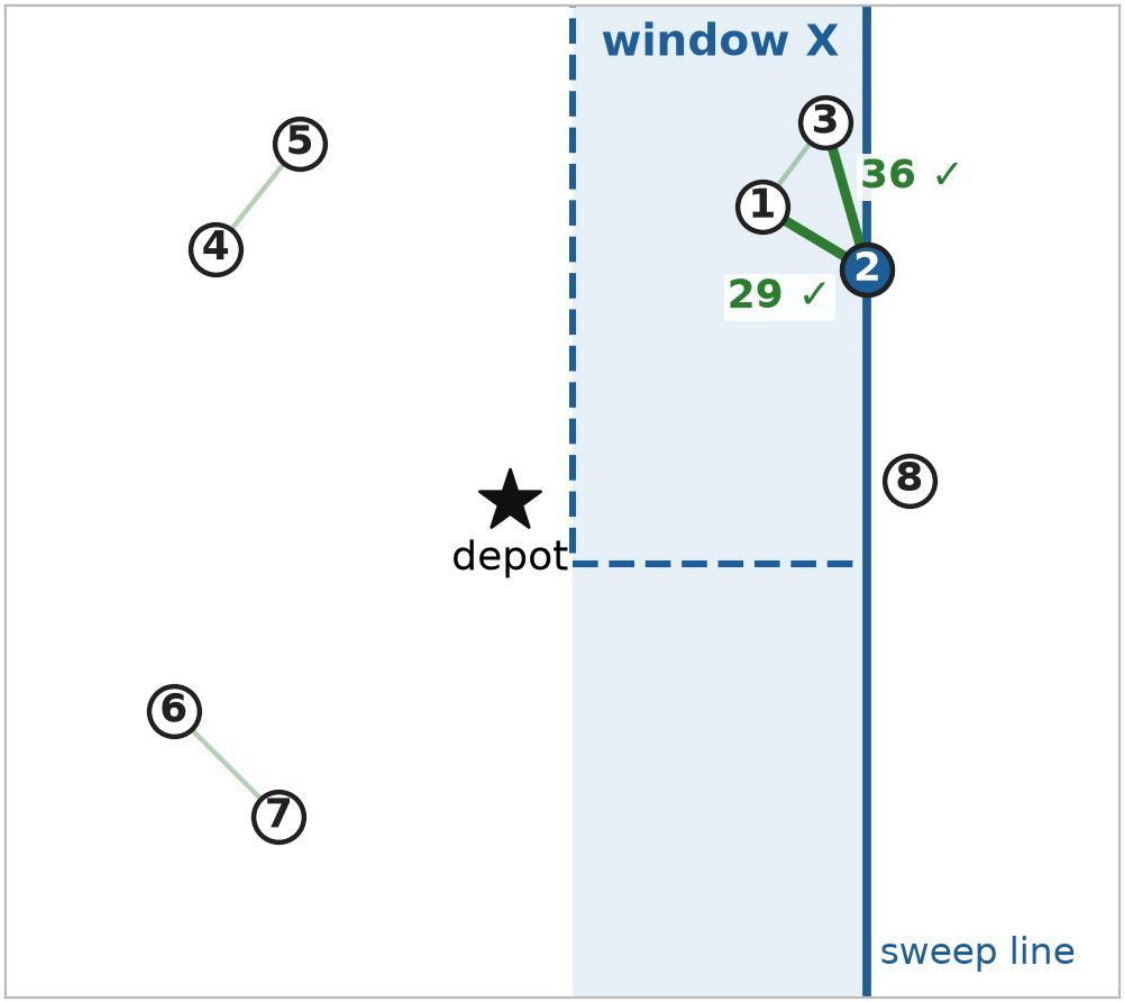
PRUNE

Line sweep keeps only short candidate edges

Sweep customers left to right. Only pairs inside the window X are checked, so we never store all N^2 edges.

Time complexity

$$O(N \log N + E)$$



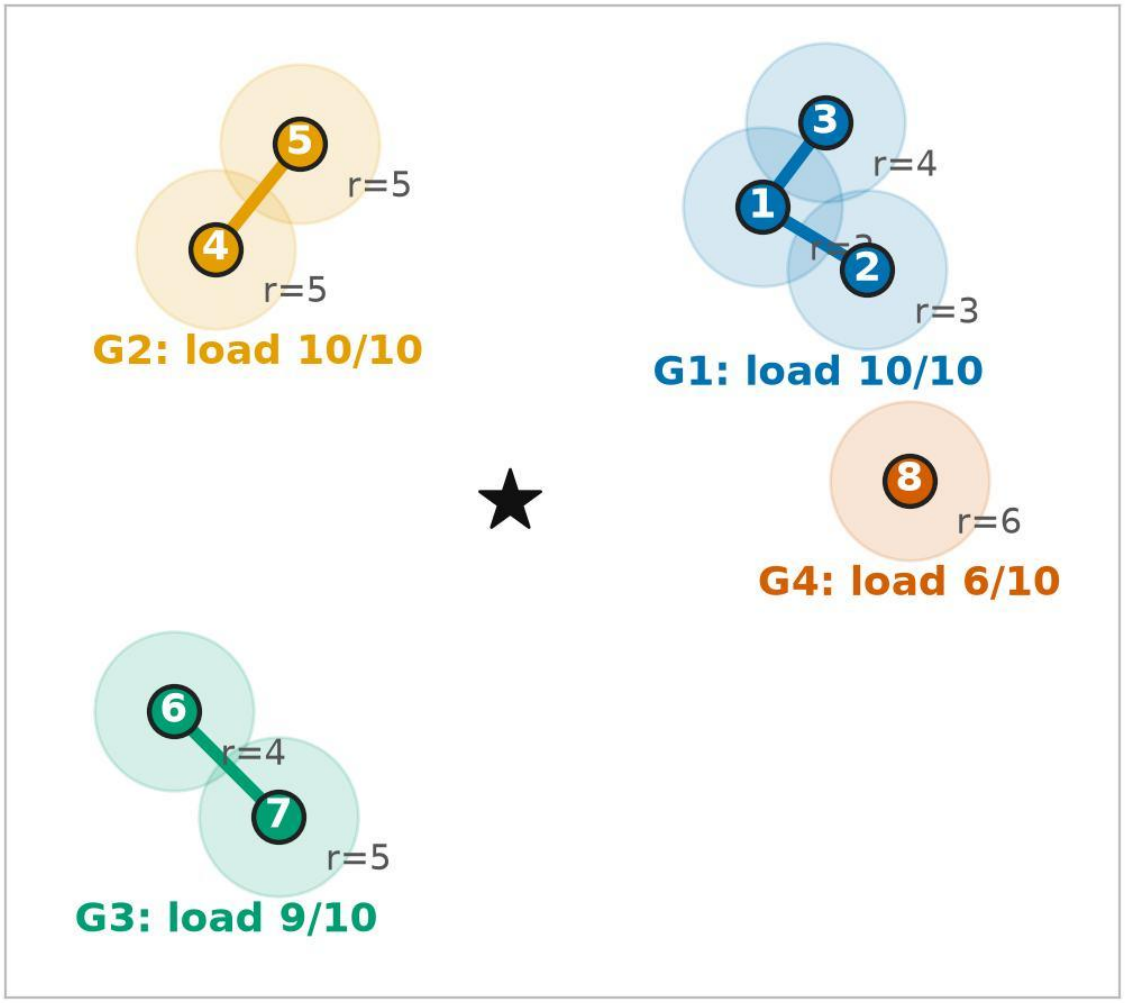
GROUP

Kruskal merges customers while demand $\leq Q$ and size $\leq X$

Edges are taken shortest first. Two groups join only if the result still fits in one trip.

Time complexity

$$O(E \log E)$$



SOLVE

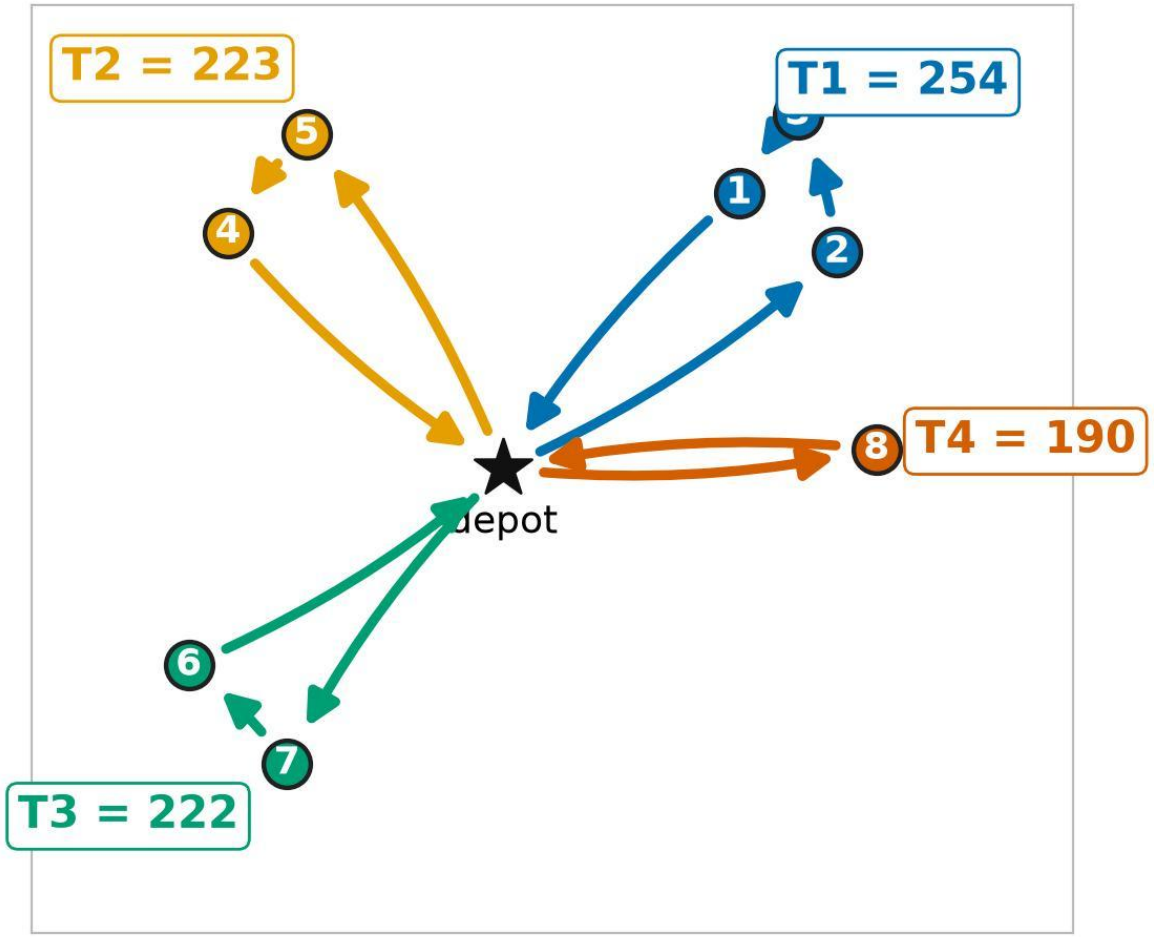
Held–Karp or Lin–Kernighan gives one tour per group

Held-Karp is exact on small groups ($X = 10$). LK scales to larger groups ($X = 40$).

Time complexity

$O(G\,X^2\,2^X)$ Held-Karp

$O(G\,X^{2.2})$ Lin-Kernighan



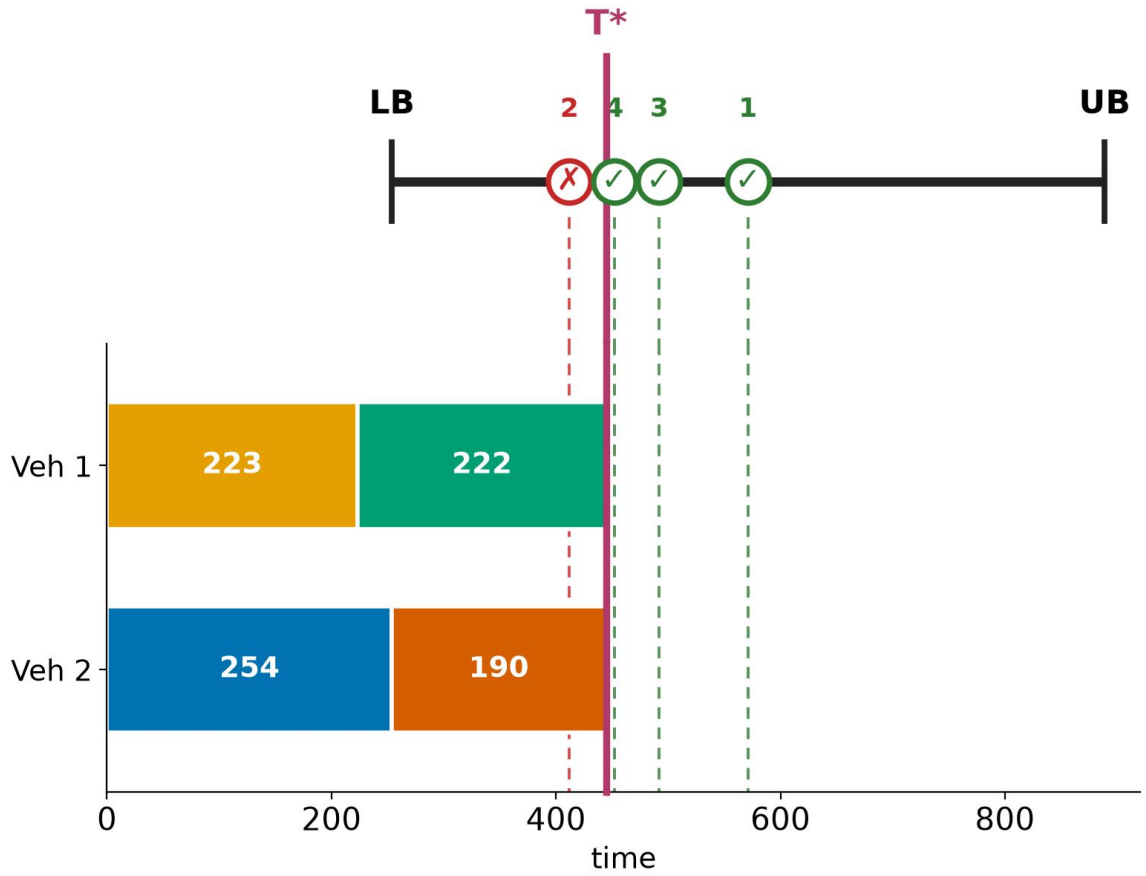
MERGE

Binary search on the makespan, pack trips onto vehicles

For each candidate M between LB and UB, check whether all trips fit on K vehicles. Keep the smallest that fits.

Time complexity

$$O(G \log G + G \log(UB - LB))$$



MERGE

Why binary search works

Makespan at most M: the trips in R can be split among K vehicles, each back by time M.

$\Sigma_{r \in Rk}\, T(r) \leq M$ for every vehicle k, where $R_k \subseteq R$ are the trips of vehicle k

If the trips fit in M, they also fit in $M + 1$: the same split still works.

Makespan	LB	…	$M^* - 1$	$\boldsymbol{M^*}$	$M^* + 1$	…	UB
Possible?	no	no	no	**yes**	yes	yes	yes

Binary search on $[LB, UB]$ finds M^*, the first "yes", where $LB = \max T(r)$ and $UB = \Sigma\, T(r)$.

MERGE

Checking one M with two pointers

Sort the trip durations D ascending (n trips). ll starts at the shortest trip, rr at the longest.

$D[ll] + D[rr] \le M$	merge: $D[rr] \mathrel{+}= D[ll]$, $ll{+}{+}$
$D[ll] + D[rr] > M$	vehicle rr is full: $rr{-}{-}$
stop when $ll = rr$	vehicles used = $n - rr + 1$

Possible $\Leftrightarrow n - rr + 1 \le K \Leftrightarrow rr > n - K$

Each index from rr to n is one vehicle R_k, and $D[rr]$ is its total duration.

Example: $D = [1, 2, 4, 5, 6, 6, 7]$, $K = 3$

Start

ll rr

index	1	2	3	4	5	6	7
value	1	2	4	5	6	6	7
	ll						rr

End, $M = 10$

index	1	2	3	4	5	6	7
value	1	2	4	5	6	10	10
				$ll = rr$			

$rr = 4$: $7 - 4 + 1 = 4 > K$, so $M = 10$ is not possible.

MERGE

Example: $M = 10$ vs $M = 11$

$D = [1, 2, 4, 5, 6, 6, 7]$, $K = 3$. The two runs are identical until step 6.

Step	ll	rr	$D[ll] + D[rr]$	$M = 10$	$M = 11$
1	1	7	1 + 7 = 8	merge	merge
2	2	7	2 + 8 = 10	merge	merge
3	3	7	4 + 10 = 14	$rr--$	$rr--$
4	3	6	4 + 6 = 10	merge	merge
5	4	6	5 + 10 = 15	$rr--$	$rr--$
6	**4**	**5**	**5 + 6 = 11**	$\boldsymbol{rr--}$	**merge**
End				**4 vehicles: no**	**3 vehicles: yes**

Binary search tries $M = 19$ yes, 12 yes, 9 no, 10 no, 11 yes, so $M^* = 11$.
Lower bound $\lceil 31 / 3 \rceil = 11$, so $M^* = 11$ is optimal for this example.

Step 6

5 + 6 = 11

Too big for $M = 10$: a 4th vehicle is needed.

Fits in $M = 11$: 3 vehicles are enough.

4 Setup

Instances (random)

Depot at (0, 0)

x_i, y_i are integers in $[-100, 100]$

q_i is an integer in $[1, 10]$

One instance per cell, one run per method, sequentially

Two grids, 8 × 8 cells

N–K with $Q = 320$

N–Q with $K = 20$

Each grid varies two parameters over 8 values; a cell is one pair of values, i.e. one random instance.

Methods and metrics

Held-Karp ($X_{max} = 10$)

LK ($X_{max} = 40$)

PyVRP (ILS), stops after 20,000 iterations without improvement

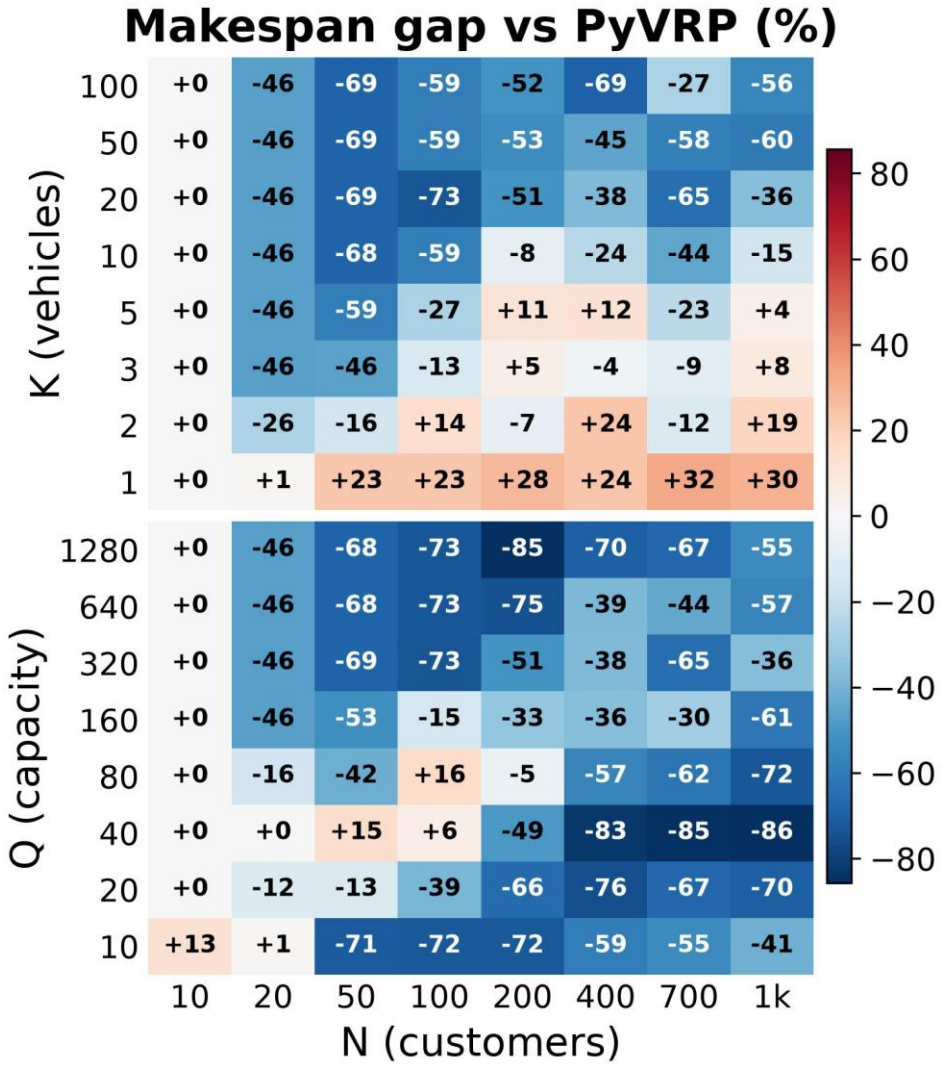
Makespan T^* (lower is better)

Wall-clock time (s)

Hardware and Libraries

Intel Core Ultra 7 155H (22 logical CPUs), 15 GiB RAM, Ubuntu 24.04 (WSL2). Python 3.13.13, NumPy 2.5.4, PyVRP 0.14.1, lk-heuristic 0.0.4.

5 Makespan vs PyVRP



85 / 128

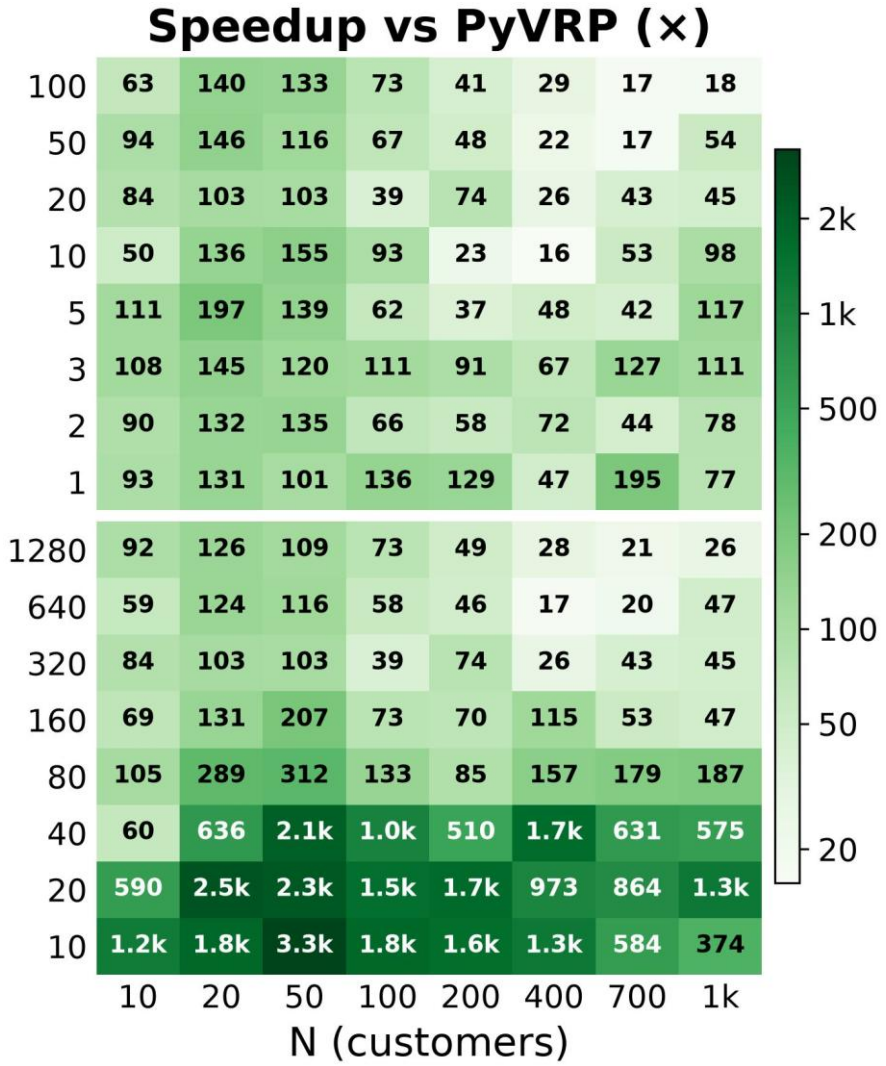
cells where our makespan is lower than PyVRP

Gap = (ours – PyVRP) / PyVRP, using the better of Held-Karp and LK.

Blue means we win, **red** means PyVRP wins.

Each cell is one instance (no averaging).

5 Speedup vs PyVRP



≥ 15×

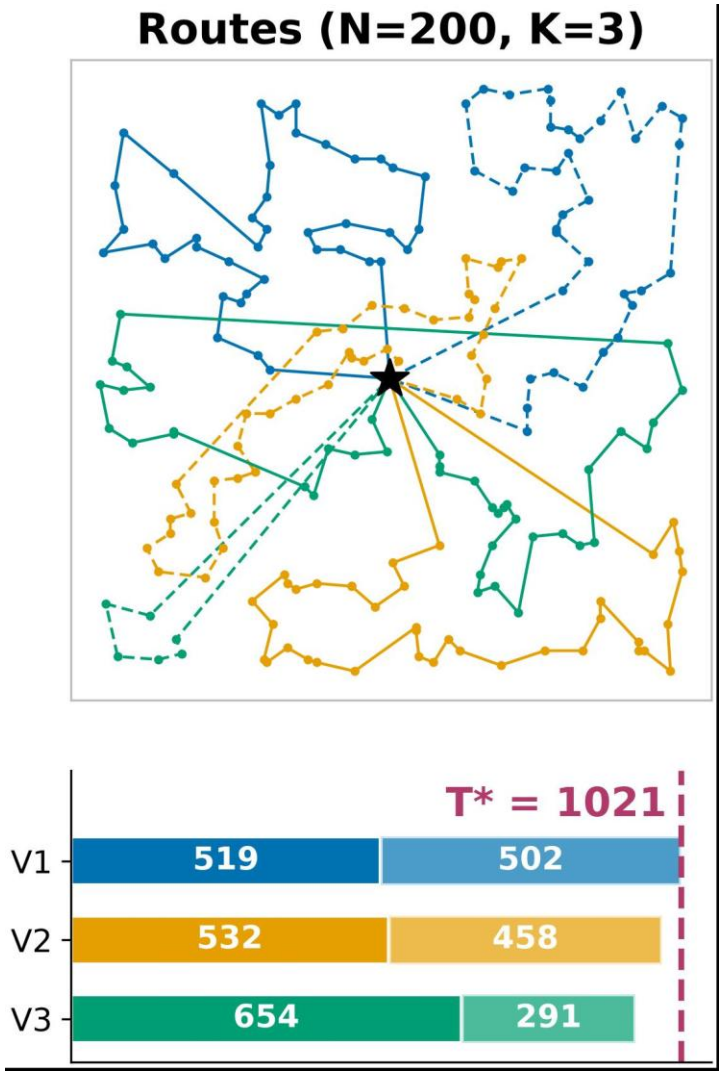
faster in every cell where we win

103×

median speedup in those cells

Speedup = PyVRP time / our time. Darker is faster.

5 Routes for $N = 200, K = 3$



$$T^* = 1021$$

makespan of the final solution

Each color is one vehicle, and each vehicle runs two trips.

The bars show each vehicle's two trip durations back to back. The last one returns at $T^* = 1021$.

Key Findings

85 of 128

cells where the better of Held-Karp and LK beats PyVRP (ILS) on makespan

≥ 15×

faster than PyVRP in all of those cells

103×

median speedup

PyVRP is better mainly with 1–5 vehicles, where its objective (total distance) is closest to the makespan.

Each cell is one random instance (no averaging).

6 Limitation

Two groups may fit in one trip ($\sum q_i \leq Q$) and finish sooner than two separate trips. Our method cannot merge trips this way.

Exact methods (e.g. dynamic programming over subsets) only scale for small N (≤ 20), so for larger N the gap to the optimum is unknown.

7 Future Works

Merge trips into one when the capacity allows and the makespan improves.

Compare with a baseline that minimises makespan, since PyVRP minimises total distance.

8 References

[1] A. Laaksonen, *Competitive Programmer's Handbook*, 2018. [Online]. Available: https://cses.fi/book/book.pdf

Code and Data



github.com/mie-intel/ehime-works

Thank you

Questions and feedback are welcome.